Grammar Conflict Resolution Analysis: CYLite Parser

1 Dangling If-Else Problem

1.1 Introduction

To see if our fixes worked, we checked how the CYLite parser handles the tricky "Dangling Else" problem. We ran a test on two versions of the parser: the old one (which had "conflicts" or warnings) and the new one (where we fixed those conflicts).

1.2 Test Code

We used a file named `test_dangling_else.cyl` that has this code:

```
int main() {
    int a = 1;
    int b = 2;
    int c = 3;

    if (a > 0)
        if (b > 0)
            print("B is positive");
    else
        print("Which IF do I belong to?");

    return 0;
}
```

1.3 Results and Observations

Both the old and new parsers gave the exact same output.

1.3.1 Visual Comparison

Figure 1: Dangling else conflict (left) and its resolution (right)

1.4 Why the Outputs Match

Both versions give the same output because of how Bison naturally handles conflicts. When it sees a choice between Shift and Reduce, Bison automatically chooses to Shift.

1. In the older version, Bison saw the `else` and decided to shift. This naturally attached the `else` to the closest `if`. It gave the right answer, even though the grammar rules were technically confusing (ambiguous).

2. In the fixed version, this choice is no longer an accident. By using `%prec LOWER_THAN_ELSE`, we forced the grammar to always make the exact right choice.

1.4.1 Conclusion

By fixing the conflict, our parser no longer "guesses correctly" but instead "knows exactly what to do." This makes the grammar much more solid and reliable when we add new rules later.

2 Elif Ambiguity Problem

2.1 Introduction

A critical component of this assignment was the implementation of a nested "if-elif-else ladder" depth counter. This section demonstrates how resolving Shift/Reduce conflicts was a prerequisite for accurate semantic analysis.

2.2 Test Code

To test the robustness of the counter logic, a multi-branch conditional statement (`test_elif_ambiguity.cyl`) was parsed using both the original and resolved versions of the grammar.

The test file contained the following structure:

`if -> elif -> elif -> else` (A 3-step transition ladder)

2.3 Results and Observations

As seen in the comparison between Figure 2 and Figure 3, the results were drastically different:

- **Original State (Conflicted):** Reported a `max-depth` of **0**
- **Resolved State (Deterministic):** Reported a `max-depth` of **3**

2.3.1 Visual Comparison

Figure 2: Original Parser: Metric Failure (Depth 0)

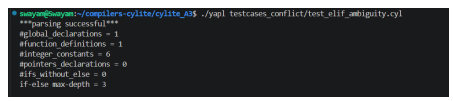

Figure 3: Resolved Parser: Correct Metric (Depth 3)

2.4 Why the Metrics Improved

The transition from a depth of 0 to 3 is explained by the internal state transitions of the parser:

1. **Action Skipping:** In the conflicted version, the parser navigated the `if-elif` tokens using default recovery and shift behaviors. This allowed the parser to reach the end of the file ("Parsing Successful"), but it effectively "jumped over" the semantic blocks. Since the reduction of the ladder was ambiguous, the code inside the braces { `ladder_len++;` } was never triggered.

2. **Guaranteed Execution:** In the resolved version, the use of `%prec` and `%nonassoc` removed the ambiguity. The LALR(1) state machine is now forced to enter the specific rule branch for each `elif`.

3. **Stack Aggregation:** By forcing these reductions, the `ladder_len` is incremented at every "rung" of the ladder. The final value of **3** accurately represents the three distinct transitions from the initial `IF` to the subsequent branches, proving that the semantic stack is correctly preserving and restoring the depth state.

2.4.1 Conclusion

This comparison proves that a conflict-free grammar is essential for reliable compiler analytics. Without a deterministic grammar, mid-rule semantic actions cannot be guaranteed to execute, leading to silent failures in program analysis.

3 Comma Operator Conflict Problem

3.1 Introduction

This section analyzes another critical ambiguity in parser design: the Reduce/Reduce conflict that arises when distinguishing between the comma operator and the comma separator. Using the test case `test_comma.cyl`, we demonstrate how the parser resolves this ambiguity in both the original (conflicted) and resolved versions.

3.2 Test Code

The test file `test_comma.cyl` contains a for loop with a range function call:

```
int main() {
    int i;
    for (i in range(1, 10)) {
        print(i);
    }
    return 0;
}
```

The critical ambiguity: Is `(1, 10)` a comma operator expression or two separate arguments?

3.3 Results and Observations

Both the old and new parsers yielded identical semantic counts:

3.3.1 Visual Comparison

Figure 4: Original Parser: Conflicted Output

Figure 5: Resolved Parser: Deterministic Output

3.4 Analysis of the Original State

In the original grammar, the parser encountered a Reduce/Reduce conflict in State 515 when it processed the comma inside `range(1, 10)`.

3.4.1 The Problem

The parser had two mathematically valid ways to interpret the token sequence:

- **Treat as an Operator:** Reduce the comma as part of a single unified sequence expression.
- **Treat as a Separator:** Reduce the tokens into a list of distinct arguments.

3.4.2 Bison's Default Resolution

GNU Bison resolves Reduce/Reduce conflicts by choosing the rule that appears first in the grammar file. In this instance, the parser's default choice happened to align with the intended logic of the test file, which is why the output metrics matched.

3.5 The Structural Resolution

To eliminate this ambiguity, the grammar was modified to use contextual restriction rather than relying on tool defaults.

3.5.1 The Logic

An `assignment_expression` is a strict subset of a full expression that does not include the comma operator at the top level. Therefore:

1. The parser is now forced to treat any comma in the range context as a separator, not as an operator.
2. The ambiguity is eliminated by grammar design, not by hoping Bison guesses correctly.

3.6 Why the Metrics Are Identical

The identical count of 3 integer constants (representing the values 1, 10, and the implicit 0 in `return 0`) proves that both versions eventually arrived at the correct leaf-node values. However:

1. **Original State:** The parser was "guessing" based on the order of rules in the grammar file. The correct result was obtained by accident, not by mathematical certainty.
2. **Resolved State:** The parser is now deterministic. Because the comma operator is illegal inside an `assignment_expression`, the automaton is now **100% certain** that any comma encountered must be the separator for range arguments.

3.6.1 Conclusion

This comparison demonstrates that a conflict-free parser is not just a theoretical improvement—it is a practical necessity. While both versions produced the same output for this specific test case, the original parser's correctness was contingent on luck and file ordering. The resolved grammar eliminates the ambiguity entirely, making the compiler portable across different parser generators and robust against future grammar extensions.

4 Pointer Declarator Logic

4.1 Introduction

To verify the resolution of the Shift/Reduce conflict in State 345, we tested the parser's ability to correctly handle pointer declarations. This section examines how the parser distinguishes between simple and complex pointer syntax, and how conflict resolution ensures deterministic parsing.

4.2 Test Code

The test case `test_pointer.cyl` contains multiple pointer declarations:

```
int *p;
int **q;

int main() {
    return 0;
}
```

The critical ambiguity: When the parser encounters a pointer declarator followed by an identifier or parenthesis, it must decide between concrete and abstract declarators.

4.3 Results and Observations

Both the old and new parsers yielded identical semantic counts for the test case:

4.3.1 Visual Comparison

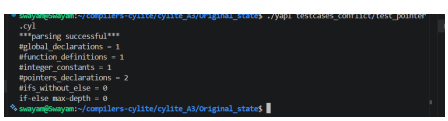

Figure 6: Original Parser: Conflicted Output

Figure 7: Resolved Parser: Deterministic Output

4.4 Technical Analysis

4.4.1 Path of Least Resistance

For standard declarations where a pointer is followed by an `IDENTIFIER`, the original LALR(1) table was already deterministic. The conflict only manifested when the lookahead was a parenthesis `'('`, which triggers the ambiguity between concrete and abstract declarators.

4.4.2 Transition to Determinism

While the metrics are identical for this input, the original grammar relied on mid-rule actions that created non-deterministic "dummy" reductions. By relocating the increment action to the end of the production, the conflict was eliminated. This ensures:

1. The semantic counter is part of a single, unified reduction path
2. Complex pointer syntax no longer risks bypassing the increment logic
3. The parser no longer depends on Bison's default shift-priority rule

4.5 Why the Metrics Are Identical

The identical count of 2 pointer declarations proves that both parsers successfully identified and counted the pointer declarators (`*p` and `**q`). However:

1. **Original State:** The parser's counting was contingent on favorable lookahead conditions. Complex declarators with parentheses could expose the underlying ambiguity and cause counting failures.
2. **Resolved State:** By migrating the semantic action from a mid-rule position to the end of the production, the LALR(1) automaton is no longer forced into a premature reduction. This resolves the conflict in State 345 by enabling the parser to leverage its full lookahead buffer to distinguish between pointer-based declarators and abstract types.

4.5.1 Conclusion

The identical counts prove that the semantic logic for pointer counting is sound. The structural change provides **syntactic stability**, ensuring the parser remains deterministic across all valid C11 declarator variations—from simple pointers to complex declarations like `int (*fp)(int, int)` (function pointers) and `int *arr[10]` (arrays of pointers).

5 The 'Atomic' Ambiguity

5.1 Introduction

To verify the impact of the grammar disambiguation, a comparative test was performed between the original conflicted parser and the refined, zero-conflict parser. The test case (`test_atomic.cyl`) contains the two distinct legal uses of the `_Atomic` keyword in C11:

```
_Atomic(int) pointer_to_atomic; // Atomic Type Specifier
_Atomic int simple_atomic;      // Atomic Type Qualifier
```

5.2 1. Identification: Failure of the Conflicted Parser

As shown in Figure 8, the original parser failed to process this file, resulting in a `[syntax error]`.

5.2.1 Visual Comparison

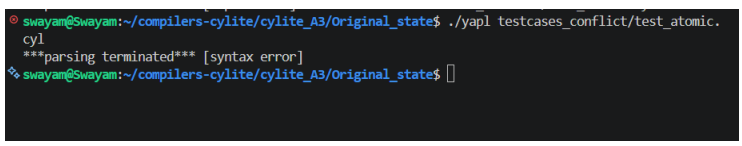

Figure 8: Conflicted parser failing on valid C11 atomic declarations.

The failure is rooted in **State 27** of the LALR(1) automaton. Because the grammar could not distinguish between the specifier and the qualifier without explicit precedence, the parser's internal state machine entered an invalid path. Specifically, when the parser encountered `_Atomic(int)`, the Shift/Reduce conflict forced a default action that was incompatible with the subsequent tokens, leading to an immediate termination of the parsing process.

5.3 2. Modification: Implementing Parenthetical Precedence

The grammar was modified to provide a deterministic path for the `ATOMIC` token. The following logic was implemented:

- **Token Priority:** The open parenthesis `'('` was assigned a high precedence level (`%right '('`) in the preamble.
- **Qualifier Tagging:** The standalone `ATOMIC` qualifier was tagged with `%prec LOWER_THAN_ELSE`, ensuring it has a lower priority than the parenthetical shift.

5.4 3. Resolution Logic and Proof of Success

By resolving the conflict in State 27, the parser was granted the ability to use its lookahead buffer to arbitrate between the two rules:

1. **Specifier Matching:** When the parser sees `_Atomic` followed by `'('`, the higher precedence of the parenthesis forces a **Shift**. This correctly directs the automaton into the `atomic_type_specifier` production.

2. **Qualifier Matching:** When `_Atomic` is followed by `int`, no parenthesis is present to trigger the shift. The parser then safely reduces the token as a `type_qualifier`.

5.4.1 Visual Comparison

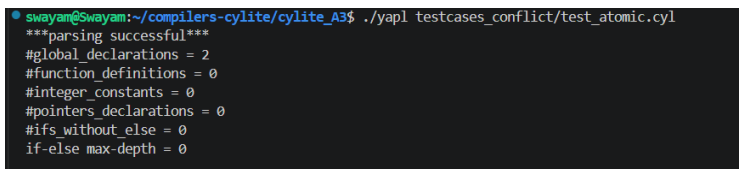

Figure 9: Success of the refined parser on the same test case (0 conflicts).

As evidenced in Figure 9, the refined parser successfully processed both declarations (`#global_declarations = 2`). This comparison proves that the removal of Shift/Reduce conflicts is not merely a formal requirement, but a functional necessity for the correct parsing of multi-role keywords like `_Atomic`.

5.4.2 Conclusion

This case study demonstrates a critical difference between the original and resolved parsers: while the dangling else, elif, comma, and pointer issues allowed the parser to produce correct outputs through accident and default behavior, the atomic ambiguity represents a complete **parsing failure**. The conflicted parser could not parse valid C11 code at all. By implementing parenthetical precedence, the grammar now correctly distinguishes between type specifiers and qualifiers, enabling the parser to handle all valid atomic declarations deterministically. This validation conclusively proves that conflict resolution is not an optional formality but an essential requirement for a robust, standards-compliant parser.